

SOR: GNU-LINUX

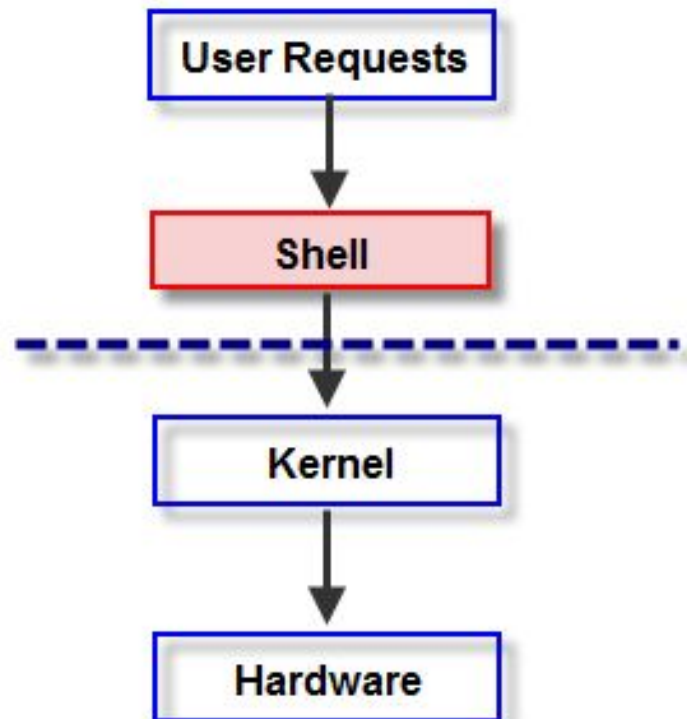
UNGS

TEMARIO

- Shell
- Comandos de la Shell
- Editores VI/VIM, nano, emacs, gedit
- Ejercicios

Shell

Shell (ó intérprete de línea de comandos) es un programa que permite al SO interpretar los comandos introducidos por el usuario.



¿Cuál es mi shell?



> echo \$SHELL
debería aparecer /bin/bash
ó la shell que estén usando

Shell - la definición más precisa

6. The Shell

For most users, communication with UNIX is carried on with the aid of a program called the Shell. The Shell is a command line interpreter: it reads lines typed by the user and interprets them as requests to execute other programs. In simplest form, a command line consists of the command name followed by arguments to the command, all separated by spaces:

`command arg1 arg2 · · · argn`

Sacado de :

The UNIX Time-Sharing System. D. M. Ritchie K. Thompson 1984

Shell

Shell's más importantes son:

- Bourne shell (sh), Creado por Stephen Bourne, es uno de los más utilizados en la actualidad. Su símbolo del sistema es \$. Es el shell estándar y el que se monta en casi todos los sistemas GNU/Linux.
- Bourne -Again shell (bash), creado para usarlo en el proyecto GNU. BASH, por lo tanto, es un shell o intérprete de comandos GNU que incorpora la mayoría de distribuciones de GNU/Linux. Es compatible con el shell sh.



Stephen Bourne

“Go away or I will replace you with a very small shell script”

Shell

¿Cómo cambio de shell?



> chsh -s /bin/sh
y reiniciar la máquina

```
~ mkdir zshhh
~ cd zshhh
~/zshhh git init
Initialized empty Git repository in /home/michiel/zshhh/.git/
~/zshhh master touch zshhh.txt
~/zshhh master gaa
~/zshhh master + gcam "first commit"
[master (root-commit) 40ef851] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 zshhh.txt
~/zshhh master Zsh is great!
zsh: command not found: Zsh
✗ ~/zshhh master
```

Ohmyzsh, un shell moderno y de moda

Comandos básicos

- **man** muestra el manual de un comando.
\$ man cat
- **cd** permite moverse entre directorios.
\$ cd /home/usuario
\$ cd ..
- **ls** lista el contenido de un directorio.
\$ ls /home/usuario
\$ ls -l
- **pwd** muestra por pantalla el path actual.

Visualización

- **cat** lista un archivo de principio a fin
`$cat file.txt`
- **more** paginador clásico de Unix para mostrar archivos por partes.
`$more file.txt`
- **less** paginador más moderno, típico de Linux para mostrar archivos por partes.
- **tail** muestra un archivo desde su final, cierta cantidad de líneas hacia arriba.
`$tail file.txt -n 5`
- **head** muestra los archivos desde su comienzo

Archivos / Directorios

- **touch** permite crear un archivo.
- **mkdir** permite crear un directorio con un nombre cualquiera.
- **cp** equivalente a copiar/pegar y cortar/pegar; con **mv** también se puede cambiar el nombre de un archivo.
- **rm** permite eliminar archivos y directorios uno a uno o en forma recursiva.
- **rmdir** permite eliminar un directorio.

Búsqueda

- **find** busca dentro del árbol de directorio.

`$find / -name file1`

- **grep** búsqueda (matching) de cadenas en archivos.

`$grep error /var/log/messages`

- **echo** escribe por la salida estándar lo que se pase por parámetro.

- **whereis** mostrar la ubicación de un fichero binario, de ayuda o fuente.

`$whereis halt`

En este caso pregunta dónde está el comando 'halt'.

Entrada y Salida Estándar

Por defecto, toda instrucción en Linux tiene tres canales básicos por donde se transmite la información, a estos canales se les denomina como los canales de entrada, de salida y de error.

Nombre	Descriptor de fichero	Destino por defecto
entrada estándar (stdin)	0	teclado
salida estándar (stdout)	1	pantalla
error estándar (stderr)	2	pantalla

La **entrada estándar** representa los datos que necesita una aplicación para funcionar.

La **salida estándar** es la vía que utilizan las aplicaciones para mostrar información. El **error estándar** es la forma en que los programas informan sobre los problemas que pueden encontrarse al momento de la ejecución

Todos estos tipos son representados físicamente como archivos en el sistema, todo en Linux son archivos.

Redireccionar I/O

- **\$ ls -l > archivo.txt**
- **\$ wc < archivo.txt** (wc , nos indica el número de líneas o palabras de un archivo)
- **\$ ls -l ~ >> archivo.txt**
- **\$ cat archivo.txt 2 > error.txt**
- **\$ cat archivo.txt 1 > log.txt**

Pipes y Salida Estándar

Un **PIPE** o tubería en Linux no es más que una forma conectar la salida estándar de un programa con la entrada estándar de otro. Esto se logra usando el símbolo | (pipe).

- **\$ top | grep firefox**
- **\$ ls /usr/bin | tail**
- **\$ ls /usr/bin | head**

VI/VIM

VI es el editor por defecto de los sistemas Unix (1974);

Actualmente lo tenemos en todas las distribuciones GNU/Linux
tiene una versión mejorada denominada VIM.

Su uso básico es el siguiente:

- `$vi <archivo>` edita un archivo en particular.
- `Esc` sale del modo edición, para ingresar comandos.
- `i` ingresa al modo edición.
- `:wq` guardar y salir.

VI/VIM

PROGRAMMER'S LIFE

```
string sender;  
sender = "Muriel Godoi";
```



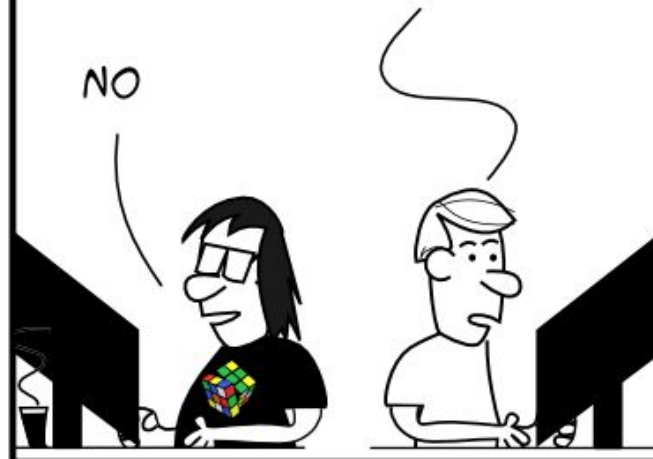
#39

I DEVELOPED A NEW
WAY TO GENERATE
RANDOM STRINGS...



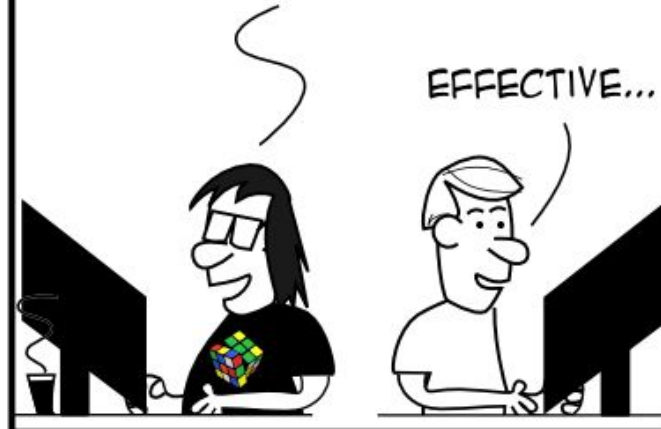
HOW? IT'S BASED
ON PROCESSOR'S
CLOCK?

NO



JUST LET THE
TRAINEE USING THE "VI"
AND ASKED HIM
TO CLOSE IT...

EFFECTIVE...



NANO

Nano es un editor de terminal para sistemas GNU/Linux es más amigable que Vi.

Su uso básico es el siguiente:

- `$nano -c <archivo>` edita un archivo en particular (y muestra el numero de linea).
- `ctrl + x` para salir,
pregunta si desea guardar y con qué nombre

GEDIT

Gedit es un editor de interfaz gráfica para sistemas GNU/Linux

Su uso básico es el siguiente:

- `$gedit <archivo>` edita un archivo en particular.
- `&` con el parámetro ampersand no se bloquea la terminal

Ejecutar en background

Agregando un `&` al final de un comando, el mismo se ejecuta en background.

```
$ sudo apt-get update &
```

Cualquier tarea que se esté ejecutando se puede enviar a background de las siguientes maneras:

La combinación '**CTRL+Z**' , suspende el job en ejecución.
Con el comando **bg**.

El comando **fg**, trae al frente el job

jobs lista los procesos que se están ejecutando en segundo plano

Si tenemos varios procesos en segundo plano podemos seleccionar el que necesitamos traer al frente con el comando **\$ fg %3** (nro proceso).

EMACS

Emacs es un super-editor extensible tanto de terminal como de interfaz gráfica para sistemas GNU/Linux que fue escrito por Richard Stallman. Sigue vigente desde su lanzamiento en 1976.

Su uso básico es el siguiente:

- `$emacs <archivo>` edita un archivo en particular.
- `alt w` para copia
- `ctrl y` para pegar
- `ctrl x + ctrl c` para salir



PROCESOS

- **top** muestra las tareas de linux usando la mayoría cpu.
\$ top
- **htop, ps**
- **kill** se utiliza para matar un proceso
\$ kill -9 8893 (pid)

Manejo de Archivos

- **stat** nos muestra una información sobre el archivo
\$ stat file.txt

```
himanshu@himanshu:~$ stat somefile
  File: 'somefile'
  Size: 52          Blocks: 8          IO Block: 4096   regular file
Device: 801h/2049d Inode: 7084498       Links: 1
Access: (0611/-rw---x--x)  Uid: ( 1000/himanshu)   Gid: ( 1000/himanshu)
Access: 2014-07-03 23:23:23.800663762 +0530
Modify: 2014-07-03 23:22:49.768495005 +0530
Change: 2014-07-03 23:22:49.816495243 +0530
 Birth: -
himanshu@himanshu:~$
```

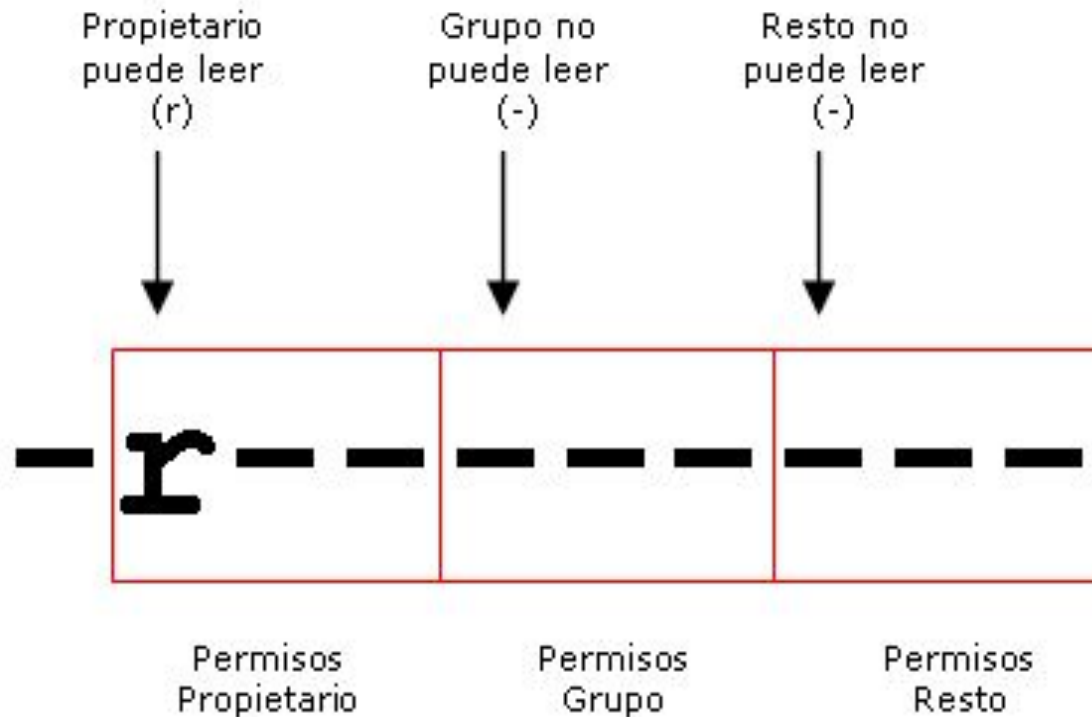
- **chown** permite cambiar el propietario de un archivo o directorio.
\$chown <nuevouser> archivo1 [archivo2 archivo3...]
\$chown -R <nuevouser> <directorio> (Rekursivo)

Permisos

En Linux cada recurso pertenece a un usuario, y a un grupo de usuarios.

`$chmod u+rw error.txt`

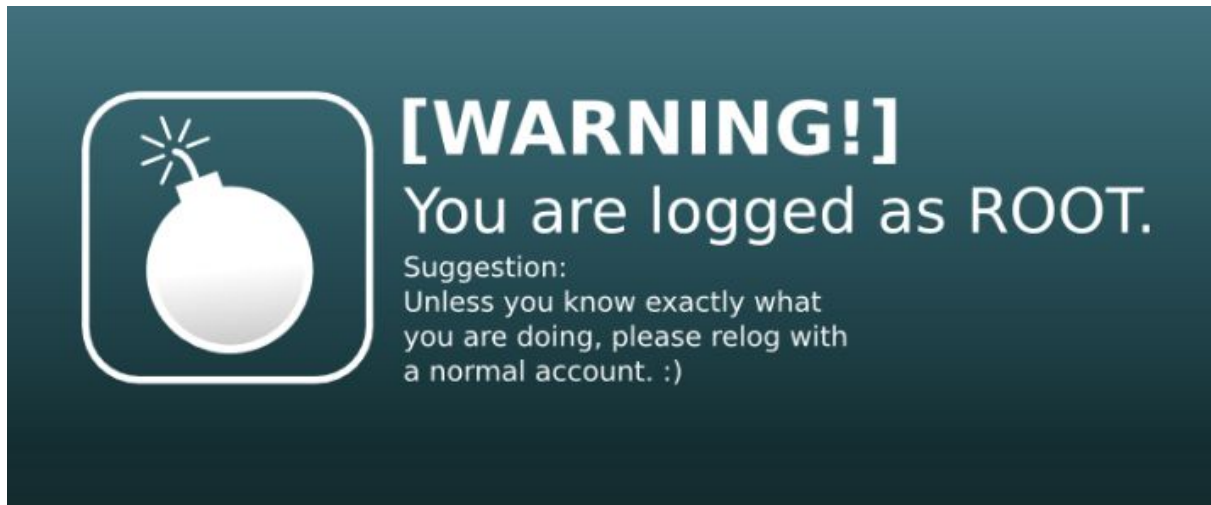
`$chmod g-r-w+x,o+rw-x error.txt`



x	---	x	---	x	-----	x
	rw		7		Lectura, escritura y ejecución	
	rw		6		Lectura, escritura	
	r-x		5		Lectura y ejecución	
	r--		4		Lectura	
	-wx		3		Escritura y ejecución	
	-w-		2		Escritura	
	--x		1		Ejecución	
	---		0		Sin permisos	
x	---	x	---	x	-----	x

Usuario Root

- También llamado superusuario o administrador.
- Su UID (User ID) es 0 (cero).
- Es la única cuenta de usuario con privilegios sobre todo el sistema.



sudo

El comando **sudo** permite a cualquier usuario ejecutar un comando como root. Solo aquellas cuentas de usuarios que pertenezcan al grupo **sudo** pueden ejecutarlos. Generalmente definidas en el archivo **/etc/sudoers**.

```
$ sudo apt-get update
```

El usuario root agrega a otros usuarios a sudoers:
root:\$ adduser usuario sudo

Su

El comando su permite loguearse como otro usuario,
usuario:\$ su otroUsuario

En particular si el usuario pertenece a sudoers se puede loguear como root:
usuario\$ sudo su
root\$

Por seguridad, **es siempre mejor trabajar como un usuario normal en vez del usuario root**, y cuando se requiera hacer uso de comandos solo de root, utilizar el comando sudo.

Grupos y Usuarios

Qué grupos existen en mi SO?

```
$ cat /etc/group
```

Que usuarios existen en mi SO?

```
$ cat /etc/passwd
```

useradd agregar un usuario

```
$ useradd -c "Juan Perez Hernandez" juan
```

userdel elimina un usuario

```
$ userdel sergio
```

passwd cambiar pass de un usuario

```
$ passwd sergio
```

Otros Comandos

- **who** quién está logueado.
- **uptime** cuánto tiempo lleva encendido el sistema.
- **uname** que kernel de Linux se está ejecutando.
- **cat /proc/interrupts** mostrar las interrupciones.
- **cat /proc/cpuinfo** contiene detalles sobre núcleos de CPU individuales.
- **exit** para terminar una sesión

Ejercicio HelloWorld:

- Realizar un script bash que pida al usuario:
su nombre
y muestre en pantalla el mensaje “Hola <nombre pasado por el usuario>”

Que comandos necesita para dar permisos de ejecución a su script y ejecutarlo?

Ejercicio 2:

Investigar qué hacen los siguientes scripts:

[Ejemplo Bash - Supermenu](#)

[Ejemplo Bash - CrearUsuarios](#)

[Ejemplo Bash - Ejecutador de Comandos](#)